IT UNIVERSITY OF COPENHAGEN

Final Report

Master of Science in Computer Science
IT-University of Copenhagen

Course Name: DevOps: Software Evolution and Software Maintenance
Course Code: KSDSESM1KU
Submission Date: ??/??/2025

Author	**Email**
Jacob Sonne	json@itu.dk
Renate Mekere	rmek@itu.dk
Adam Nørgård Aabye	aaab@itu.dk
change file name to `MSc_group_e.pdf` author5	author4
author6	author5
	author6

Table of contents

1 Process' perspective 1
1.1 CI/CD pipelines 1
1.2 Monitoring 1
1.3 Logging 1
1.4 Security 2
1.5 Availability and scaling 2
2 System's Perspective 3
2.1 Design and Architecture 3
2.2 Dependencies 3
2.3 Current System State 4

1 Process' perspective

1.1 CI/CD pipelines

TEST: pipeline is working Our project mainly use two separate pipelines called "PR Quality Gate" and "CI/CD Pipeline Production Server" LINK

PR Quality Gate pipeline

This pipeline functions as a quality gate whenever we create a Pull Request (PR) into our development branch called "develop". Each time a PR is created or updated, the pipeline would run some static analysis checks, build the solution, and test if our testsuite passes. CREATE DIAGRAM.

CI/CD Pipeline Production Server pipeline

This pipeline also runs the quality checks as the "PR Quality Gate" pipeline, however after that it also pushes our docker images to a docker registry and deploys our application in our production environment. The pipeline is triggered each time an update is triggered on the main branch. CREATE DIAGRAM.

1.2 Monitoring

1.3 Logging

The solution makes use of a go-library called "zerolog". We chose zerolog as this framework enable us to handle logs at different log-levels easily, it is fast as well as used widely by other go-users.

Promtail was used to ...

Loki was used to ...

As grafana already was used to show monitoring information, we also chose to use it for seeing the aggregated logs.

1.4 Security

1.5 Availability and scaling

To achieve high Availability we use Docker Swarm with an on-failure restart policy and a rolling update strategy. Docker Swarm allows us to have multiple replicas of our services. If one replica goes down, other replicas will still be up and running, meaning that we don't suffer downtime. The on-failure restart policy means that any replica which goes down, is rebooted automatically. The rolling update policy means that when we ship new features, the replicas are updated one at a time, which means we can achieve zero down time when making new releases.

Docker Swarm is also the answer to the question of scaling. By replicating our services we employ horizontal scaling, and with Swarm Mode also comes load balancing. This means that requests to our services are distributed among the replicas to not overburden any single replicas. The application is running on three nodes (Digital Ocean Droplets). One node for the Postgres database, one swarm-manager node and one worker node. This once again means high availability since, if the worker dies the manager is still controlling the swarm and running containers, and if the manager dies, the worker node can still run existing services. Ideally we would have more nodes, and more managers to ensure that even if a manager dies, another manager can still remain in control of the swarm.

2 System's Perspective

2.1 Design and Architecture

2.2 Dependencies

The following are the dependencies for our Minitwit system, listed in an orgainized manner in order to better provide an overview.

Runtime Dependencies

These dependencies include the environment to run the application on including the operating system, VM technologies and programming language.

- Ubuntu Linux: Operating System
- Docker (swarm mode): Virtual Machine
- golang:1.25 (Docker image)

Application Libraries

This list contains the libraries and what each library is for.

- Go 1.25.6: Programming language.
- Gorilla Mux: HTTP request routing.
- Gorilla Sessions: Session management.
- GORM: Object-relational mapper.
- PostgreSQL Driver: Database connectivity.
- go-sqlite3: SQLite support for testing/development.
- Zerolog: Structured logging.
- Prometheus client_golang: Metrics instrumentation.

Testing

- Python 3.11: Programming language
- pytest: Testing framework

DevOps / Build Dependencies

This list includes tools for employing DevOps practices.

- Github: Code repository
- Dockerhub: Docker image repository
- Github Actions: CI/CD Pipeline
- Codacy: Quality assurance
- SonarQube: Quality assurance

2.3 Current System State